Architectural Evolution of the Stela Platform: Registry Federation and Agentic Governance Regimes

1. Executive Introduction: The Stela Governance Paradigm

The evolution of software engineering from purely human-centric workflows to AI-augmented and AI-driven development necessitates a fundamental reimagining of repository governance. The Stela platform, as analyzed through the provided repository artifacts and the "Agents Subsystem Validation Discussion," represents a sophisticated response to this challenge.[1] Unlike traditional Continuous Integration/Continuous Deployment (CI/CD) pipelines that enforce code quality through static analysis of syntax, Stela introduces a "Constitution-First" architecture where the governance of the AI agents themselves—the "Staffing Protocol"—is codified into the repository's file structure and operational logic.

The central thesis of the Stela architecture is that AI agents, referred to as "Contractors," must operate under the strict supervision of "Integrator" logic and human "Operator" authority.[1] This is enforced not merely through prompt engineering, but through a "Pointer-First" Agent Constitution where agents do not possess intrinsic, hallucination-prone instructions but rather point to immutable governance documents (the Policy of Truth or PoT) and Just-In-Time (JIT) skills.

This report provides an exhaustive architectural analysis of the current state of the Stela repository (HEAD 57af7cdbf) and details the implementation of **Dispatch Packet OPS-0033 (DP-OPS-0033)**. This dispatch is critical to resolving two structural gaps identified in the "Agents Subsystem Validation Discussion": **Registry Redundancy (Gap 8.1)** and the lack of **Automated Agent Testing (Gap 8.3)**.[1] The current state exhibits a "split-brain" registry model where docs/library/INDEX.md functions simultaneously as a navigation surface and a partial registry, while docs/ops/registry/AGENTS.md acts as a deep registry for agents.[1] This redundancy introduces operational friction and the risk of state drift. Furthermore, while the "Skills" subsystem enjoys robust automated harvesting and validation via ops/lib/scripts/skill.sh, the "Agents" subsystem lacks a dedicated functional testing regime beyond static linting.[1]

The following sections will deconstruct the current architecture, analyze the theoretical implications of the identified gaps, and propose a comprehensive "Federated Registry Model" and "Behavioral Testing Regime." This analysis culminates in the drafting of DP-OPS-0033, which will execute the refactoring of the registry architecture and the institution of a new

testing harness.

2. Forensic Architectural Audit (HEAD 57af7cdbf)

A rigorous forensic audit of the repository dump was conducted to establish the baseline state of the Stela platform. This audit focused on the interactions between the "Run" jurisdiction (ops/) and the "Explain" jurisdiction (docs/), specifically analyzing how the system manages the identity and capability of its AI workforce.[1]

2.1 The "Split-Brain" Registry Pathology

The primary structural deficiency identified in the current architecture is the bifurcation of the "Registry" concept. In a well-architected data system, the Single Source of Truth (SSOT) principle dictates that data should be normalized—stored in one location and referenced elsewhere. The Stela repository currently violates this principle in its management of library entities (Agents, Skills, and Tasks).

The file docs/library/INDEX.md currently serves a dual purpose that conflates two distinct concerns: navigation and registration.[1] First, it acts as a human-readable "Front Door" or navigation aid for the Operator, listing available resources in a categorized format. Second, it acts as a database for the operational scripts. The scripts ops/lib/scripts/skill.sh and ops/lib/scripts/agent.sh contain logic that greps and appends to this file directly. For example, the skill.sh script uses a function insert_skill_index_entry to inject a pipe-delimited string into INDEX.md whenever a new skill is promoted.[1]

Simultaneously, a deeper, more detailed registry exists for agents at docs/ops/registry/AGENTS.md.[1] This file contains metadata that is essential for the "Integrator" agent but too dense for the general index, such as the "Provenance" (the Dispatch Packet ID that spawned the agent) and "Specialization" details. The audit reveals that agent.sh performs "double-bookkeeping," writing to both AGENTS.md and INDEX.md.[1]

This "split-brain" architecture introduces significant operational risk. If a script execution is interrupted or if a manual edit is made to one file but not the other, the system enters a state of "Drift." An agent might be "indexed" (visible to the user) but not "registered" (recognized by the governance tooling), or vice versa. Furthermore, the absence of corresponding deep registries for Skills (docs/ops/registry/SKILLS.md) and Tasks (docs/ops/registry/TASKS.md) means that these entities rely entirely on the overloaded INDEX.md for their canonical existence, creating an inconsistent data model across the library.[1]

2.2 The Agent Testing Gap (Gap 8.3)

While the Stela platform enforces rigorous static analysis through its repo-gates workflow—utilizing scripts like tools/lint/truth.sh to check for spelling and canon integrity, and tools/lint/style.sh for markdown formatting—it lacks a functional verification layer for its most

critical component: the AI Agents.[1]

The "Agents Subsystem Validation Discussion" highlights this as Gap 8.3.[1] The current system assumes that if an agent file (e.g., R-AGENT-01.md) exists and passes syntax linting, it is operational. However, an agent definition in Stela is a complex web of "Pointers" to other files. An agent points to the Policy of Truth (PoT.md), governance documents, specific tools in ops/bin/, and Just-In-Time skills in docs/library/skills/.[1]

Without a behavioral testing regime, an agent could be defined with "Dead Pointers"—references to skills that have been deleted or tools that do not exist. For instance, if R-AGENT-02 references S-LEARN-99 and that file is removed, the agent will hallucinate instructions during runtime, potentially violating the "Absolute Literalism" standard defined in the PoT.[1] The audit of ops/lib/scripts/agent.sh confirms the presence of a "Context Hazard" check (preventing recursive loading of the agent library) but finds no logic that verifies the validity of the *outbound* pointers.[1]

2.3 Script Interdependency and Fragility

The operational scripts are currently tightly coupled to the presentation layer. The audit of ops/lib/scripts/skill.sh reveals that it hardcodes the path to docs/library/INDEX.md and relies on specific string parsing logic (awk/grep) to insert entries.[1] This makes the INDEX.md file fragile; a change in its formatting for aesthetic reasons could break the automated promotion logic of the platform.

Furthermore, the tools/lint/library.sh script, which serves as the "Library Guard," currently validates the integrity of the library by checking INDEX.md for "ghosts" (files on disk not in the index) and "dead ends" (index entries not on disk).[1] While this is a valuable check, it reinforces the reliance on INDEX.md as the database. To move to a robust "Federated Registry Model," this linter must be refactored to treat the new deep registries in docs/ops/registry/ as the Source of Truth, demoting INDEX.md to a derivative artifact.

3. Theoretical Framework: The Federated Registry Model

To resolve the structural inefficiencies identified in the forensic audit, this report proposes the adoption of the **Federated Registry Model**. This architectural pattern aligns with the "Filing Doctrine" of the Policy of Truth, which mandates a strict separation between "Run" (executable logic and data) and "Explain" (documentation and manuals).[1]

3.1 Decoupling Data from Presentation

In the Federated Registry Model, the concept of "Registration" is decoupled from

"Navigation."

The Data Layer (Registry):

The definitive record of existence for any entity in the Stela system will reside in docs/ops/registry/. These files will be structured as strict Markdown tables, optimized for machine readability (by ops/ scripts) while remaining human-readable. They serve as the "Database" of the repository.

- **docs/ops/registry/AGENTS.md**: The SSOT for Agents.
- **docs/ops/registry/SKILLS.md**: The SSOT for Skills.
- **docs/ops/registry/TASKS.md**: The SSOT for Tasks.
- **docs/ops/registry/PROJECTS.md**: The SSOT for Projects.

The Presentation Layer (Index):

The file docs/library/INDEX.md will be refactored to serve purely as a navigation aid. Instead of containing data tables, it will contain **static links** to the registries. This eliminates the "double-bookkeeping" problem entirely. The Index points to the Registry; the Registry points to the File. If a new agent is added, the script only updates the Registry. The link in the Index remains valid because it points to the Registry file, not the individual agent entry.

3.2 Data Normalization and Provenance

The creation of deep registries for Skills and Tasks allows for the capture of metadata that is currently lost or scattered. The "Agents Subsystem Validation Discussion" emphasizes the importance of "Provenance"—knowing *why* a file exists (which Dispatch Packet created it) and *when* it was captured.[1]

Currently, ops/lib/scripts/skill.sh generates a provenance block inside the skill file itself.[1] While this preserves the history within the artifact, it makes it difficult to audit the system's evolution at a high level. By adding a "Provenance" column to the new docs/ops/registry/SKILLS.md, the Integrator agent can instantly scan the registry to identify skills generated by a specific DP (e.g., DP-OPS-0033) without opening dozens of individual files. This aligns with the "Explainability-First" philosophy of Stela.[1]

3.3 Jurisdiction of the Registry

A nuance of the Stela Filing Doctrine is the distinction between ops/ and docs/. The Policy of Truth states that ops/ is for "Run" (binaries, manifests) and docs/ is for "Explain".[1] The placement of registries in docs/ops/registry/ is a deliberate architectural decision. Although these files function as data sources for the scripts, they are fundamentally *documentation of the operational state*. They explain *what* is currently operational in the system. Therefore, they belong in the docs/ tree, specifically under the docs/ops/ sub-jurisdiction, which is designated for "Checklist-heavy, precise, dry, no-fluff" documentation.[1] This placement

ensures that the "Run" layer (ops/) remains focused on *logic* (scripts/binaries) while the "Explain" layer (docs/) retains custody of *state definitions*.

4. Operational Regime: Automated Agent Testing (Gap 8.3)

The second critical gap addressed by this report is the lack of a functional testing regime for agents. The "Agents Subsystem Validation Discussion" identifies this as Gap 8.3 and suggests the implementation of tools/test/agent_behavior.sh.[1]

4.1 Theory of Agent Verification

Testing AI agents presents unique challenges compared to traditional software testing. LLMs are non-deterministic; their output can vary even with identical inputs.[2] However, the Stela "Pointer-First" architecture reduces this variability by forcing agents to adhere to a rigid structure of pointers and constraints.[1]

Therefore, the testing regime for Stela agents should focus on **Structural Determinism** rather than **Output Stochasticism**. We do not need to test *what* the agent thinks; we need to test *that the agent has the correct context to think effectively*. This is analogous to a "Linter for Logic."

4.2 The tools/test/agent_behavior.sh Harness

The proposed testing harness will enforce the following invariants for every agent defined in docs/library/agents/:

1. **Pointer Integrity (Dead Pointer Check):** The harness will parse the ## Pointers section of each agent. For every file referenced (e.g., PoT.md, TASK.md, docs/GOVERNANCE.md), it will verify the file's existence on disk. This prevents the "Dead Pointer" hallucination risk.
2. **Toolchain Validity:** The harness will parse the Authorized toolchain list. For every binary referenced (e.g., ops/bin/open), it will verify that the file exists in ops/bin/ and is executable. This ensures the agent never attempts to use tools that have been deprecated or removed.
3. **Governance Compliance:** The harness will verify the presence of mandatory constitutional sections: ## Provenance, ## Role, ## Specialization, and ## Scope Boundary. The absence of any section indicates a violation of the Staffing Protocol.[1]
4. **Skill Linkage:** The harness will parse the JIT skills list. It will verify that every referenced skill (e.g., S-LEARN-01.md) exists in the docs/library/skills/ directory.

This approach moves agent testing from a subjective "review" process to a binary "pass/fail" gate within the CI/CD pipeline (repo-gates), ensuring that no broken agent definition can ever

merge to main.

5. Detailed Implementation Plan (DP-OPS-0033)

This section details the specific file modifications required to execute the transition to the Federated Registry Model and the institution of the Agent Testing Regime. These changes form the core of the Dispatch Packet.

5.1 Registry Creation and Population

We will create two new registry files to complement the existing Agent registry.

Table 1: Schema for docs/ops/registry/SKILLS.md

ID	Name	Provenance (DP)	Friction Context	File Path
S-LEARN-01	Verification Loop	DP-OPS-0014	Payload Verification	docs/library/skills/S-LEARN-01.md
S-LEARN-02	Project Guidelines	DP-OPS-0014	Project Scaffolding	docs/library/skills/S-LEARN-02.md
...	...	...	...	...

Table 2: Schema for docs/ops/registry/TASKS.md

ID	Name	Type	File Path
B-TASK-01	Code Review	Analysis	docs/library/tasks/B-TASK-01.md
B-TASK-02	Verification Command	Validation	docs/library/tasks/B-TASK-02.md
...	...	...	...

The existing docs/ops/registry/PROJECTS.md will be updated to match this schema, adding

columns for Created, Status, and Notes to standardize the registry interface for the ops/bin/project tool.

5.2 Refactoring docs/library/INDEX.md

The INDEX.md file will be stripped of its pipe-delimited tables. It will be rewritten to serve as a high-level navigation map, strictly pointing to the registries.

- **Old Structure:**
  --- 6. Agents (Role Definitions) ---
  agent-r-agent-01 | Agent: architect | docs/library/agents/R-AGENT-01.md
- **New Structure:**
  --- 6. Agents (Role Definitions) ---
  See the(../ops/registry/AGENTS.md) for the authoritative roll call.

This change decouples the index from the data, allowing the registries to grow without cluttering the navigation.

5.3 Operational Script Refactoring

The scripts ops/lib/scripts/agent.sh and ops/lib/scripts/skill.sh must be updated to target the new registries.

ops/lib/scripts/skill.sh Updates:

- Remove the function insert_skill_index_entry.
- Implement insert_skill_registry_entry. This function will format the skill metadata into the pipe-delimited format required by docs/ops/registry/SKILLS.md and append it to that file.
- Update the cmd_promote function to call the new registry insertion function instead of the index insertion function.
- Update grep checks that prevent duplicate IDs to check the Registry instead of the Index.

ops/lib/scripts/agent.sh Updates:

- Remove insert_agent_index_entry.

- Refine insert_registry_entry to ensure it targets docs/ops/registry/AGENTS.md exclusively and correctly formats the table row.
- Verify that cmd_promote no longer touches INDEX.md.

5.4 Linter Updates

The tools/lint/library.sh script currently enforces the "Library Guard" by checking INDEX.md. This must be rewritten.

- **New Logic:** The script will iterate through each library directory (agents/, skills/, tasks/).
- **Check:** For each file found (e.g., S-LEARN-01.md), it will grep the corresponding Registry file (SKILLS.md) to ensure the ID exists.
- **Reverse Check:** It will verify that every file path listed in the Registry actually exists on disk.
  This ensures the "Ghost" and "Dead End" checks remain active but are now enforcing the new SSOT.

6. Risk Assessment and Migration Strategy

6.1 The "Ghost" Interval Risk

A critical risk during this migration is the "Ghost" interval—the moment between when INDEX.md is cleared and the new Registries are populated. If tools/lint/library.sh runs during this interval (e.g., in a CI check triggered by a partial commit), it may fail, blocking the merge.

- **Mitigation Strategy:** The Dispatch Packet mandates an atomic update. The modification of INDEX.md, the creation of the Registries, and the update of the Linters must occur in the same commit or the same Pull Request. The Target Files allowlist in the DP explicitly includes all these files to permit this atomic transaction.

6.2 Agent Testing Overhead

The introduction of tools/test/agent_behavior.sh adds a step to the repo-gates workflow.

- **Performance Impact:** The proposed tests are static (grepping and file checking). The execution time is estimated to be under 1 second, adding negligible overhead to the CI pipeline.
- **Benefit:** This low-cost check prevents high-cost runtime failures where an agent halts due to a missing tool or skill, adhering to the "Shift Left" philosophy of catching errors early.

7. Conclusion

The execution of DP-OPS-0033 marks the maturation of the Stela platform from a "Documented" system to an "Architected" system. By decoupling the Navigation Layer (Index) from the Data Layer (Registry), we enable the infinite scalability of the library without

cluttering the operator's view. Furthermore, the introduction of the Agent Testing Regime transforms the "Policy of Truth" from a passive constitution into an active, verified operational standard. This ensures that the "Staffing Protocol" remains robust even as the number of AI agents and skills grows, solidifying Stela's position as a rigorously governed AI development environment.

8. Dispatch Packet OPS-0033

The following section contains the strict TASK.md format Dispatch Packet required to execute this architecture. It is drafted with absolute fidelity to the repository's "Freshness Gate" and "Behavioral Logic Standards".[1]

DP-OPS-0033: Registry Consolidation & Agent Testing Regime

3.1 Freshness Gate (Must Pass Before Work)

Base Branch: main

Required Work Branch: work/registry-refactor-0033

Base HEAD: 57af7cdbf

Required local re-check (worker runs; paste results in RESULTS):

- git rev-parse --abbrev-ref HEAD
- git rev-parse HEAD
- git status --porcelain

Worker must already be on the Required Work Branch listed above (no branch creation or switching). STOP if any mismatch. STOP if the Required Work Branch is missing. STOP if told to create or switch branches.

3.2 Required Context Load (Read Before Doing Anything)

Worker must confirm loaded before acting:

- PoT, SoP, CONTEXT, MAP
- Plus any DP-scoped files listed here:
  - ops/lib/scripts/agent.sh
  - ops/lib/scripts/skill.sh
  - tools/lint/library.sh
  - docs/library/INDEX.md

(OPEN is for Integrator/DP Writer state refresh; Worker does not read OPEN). DISPOSABLE ARTIFACTS (chat logs, etc.) MUST NOT BE REFERENCED OR INCLUDED.

3.3 Scope and Safety

- **Objective:** Consolidate agent/skill/task registries into docs/ops/registry/, refactor INDEX.md to use static links, and institute an automated agent testing harness.
- **Non-Goals (optional, drift-killer):**
  - No changes to the content of existing agent definitions (only their registration).
  - No changes to projects/ source code.

Target Files allowlist (hard gate)

- docs/ops/registry/AGENTS.md
- docs/ops/registry/SKILLS.md (new)
- docs/ops/registry/TASKS.md (new)
- docs/ops/registry/PROJECTS.md
- docs/library/INDEX.md
- ops/lib/scripts/agent.sh
- ops/lib/scripts/skill.sh
- tools/lint/library.sh
- tools/lint/project.sh
- tools/test/agent_behavior.sh (new)
- SoP.md

Allowlist rule: exact paths only; use (new) prefix only when the DP explicitly allows new files.

- **Conditional Requirements:** If canon/governance surfaces (e.g., TASK.md) are modified, SoP.md MUST be included in the allowlist and updated.
- **Stop Condition (scope):** If any required change falls outside the allowlist, **STOP** and report.
- **Forbidden (unless DP explicitly overrides):**
  - No edits to .github/

    - No edits to projects/ unless explicitly authorized
    - No renames/moves/deletes unless explicitly authorized
- **Worker Git Authority (non-negotiable):**
    - Workers do not create/switch branches.
- Workers do not stage/commit/merge/push.
    - Workers only return diffs/results/receipt.
- **Precedence:** If any conflict exists, **PoT.md wins**.
  If unclear, STOP and ask.
- **Strict Stop Conditions (generic):**
    - STOP if any placeholder text remains in final outputs ([...], TODO, TBD, ...).
- STOP if required DP inputs are missing (do not guess).
- STOP if IDs / naming constraints in the DP are inconsistent (when applicable).

3.4 Execution Plan (A-E Canon)

3.4.1 State

- **Context:** Currently, docs/library/INDEX.md acts as a redundant registry for agents and skills, causing double-bookkeeping in ops/lib/scripts/*.sh. There is no dedicated registry for Tasks or Skills, and no automated testing for Agent pointer integrity.
- **Drift:** Registry logic is split between docs/library and docs/ops/registry. Agent definitions lack functional verification.
- **Desired State:** All entities (Agents, Skills, Tasks) are registered in docs/ops/registry/*.md. docs/library/INDEX.md serves only as navigation. tools/test/agent_behavior.sh validates agent integrity.

3.4.2 Request

Numbered tasks (concrete, no vibes):

1. **Create Registries:**
    - Create docs/ops/registry/SKILLS.md and docs/ops/registry/TASKS.md using the format | ID | Name | File Path | Notes |.
    - Populate them with the current contents extracted from docs/library/INDEX.md.
2. **Refactor Index:**
    - Modify docs/library/INDEX.md. Remove the "database" tables. Replace them with static links to the new registries in docs/ops/registry/.
3. **Update Scripts (Harvest/Promote):**
    - Modify ops/lib/scripts/agent.sh: Remove insert_agent_index_entry. Ensure it only updates docs/ops/registry/AGENTS.md.
    - Modify ops/lib/scripts/skill.sh: Remove insert_skill_index_entry. Replace with insert_skill_registry_entry targeting docs/ops/registry/SKILLS.md.
4. **Update Linters:**

    - Modify tools/lint/library.sh: It must now verify that every file in docs/library/* has an entry in the corresponding docs/ops/registry/*.md file, instead of checking INDEX.md.
    - Update tools/lint/project.sh to look up valid Agents/Tasks/Skills from the new Registry files instead of INDEX.md.
5. **Institute Testing Regime:**
    - Create tools/test/agent_behavior.sh.
    - **Logic:**
        - Iterate through docs/library/agents/*.md.
        - Verify ## Pointers exist.
        - Verify Authorized toolchain binaries exist in ops/bin/.
        - Verify ## Scope Boundary exists.
        - Fail if any check fails.
6. **Update Project Registry:**
    - Verify docs/ops/registry/PROJECTS.md aligns with the new schema. Update if necessary to match the column structure.

3.4.3 Changelog

- **Log (human-readable, 1–6 bullets):**
    - Created SSOT registries for Skills and Tasks in docs/ops/registry/.
    - Refactored docs/library/INDEX.md to be a navigation-only surface.
    - Updated agent.sh and skill.sh to write to the new registries.
    - Implemented tools/test/agent_behavior.sh for agent pointer integrity verification.
    - Updated linters to enforce registry consistency.

3.4.4 Patch / Diff

- **Format:** Unified diff (preferred) or anchored snippets.

3.4.5 Receipt (Required)

Verification (MUST RUN; or report NOT RUN + reason + risk)

- ./ops/bin/dump --scope=platform (or repo-correct equivalent when scope differs)
- If dump refiners are used, include this fallback line in the DP:
    - Fallback:./ops/bin/dump --scope=platform --format=chatgpt --out=auto --bundle
- **Zero-Byte Check:** Verify dump is not empty (test -s <path>).
- Context lint (use repo-canonical command):
    - Prefer: bash tools/lint/context.sh
    - If not found and required: **STOP** and ask (do not invent a substitute)
- Truth lint (when PoT or canon surfaces touched; use repo-canonical command):
    - Prefer: bash tools/lint/truth.sh
    - If not found and required: **STOP** and ask (do not invent a substitute)

Skill capture (required during normal DP processing)

- Workflow: run ops/lib/scripts/skill.sh harvest to auto-generate a draft with provenance and semantic collision checks.
- Review: inspect the draft in storage/handoff/ for heuristic accuracy (Hot Zone and High Churn). Refine Invocation guidance and Solution.
  Do not edit the Provenance block.
- Promote: run ops/lib/scripts/skill.sh promote <draft_path> or log "no new skill promoted" with a rationale in RESULTS.
- Legacy capture is forbidden. Do not manually write skill files; the provenance block is required.
- If a DP requires skill capture, the DP Target Files allowlist must include docs/library/SKILLS.md.
- Proof required in RESULTS:
  - Harvest-only: command invoked plus draft path and ls storage/handoff/ output.
- Promotion: command invoked plus grep -n "Promotion Packet:" docs/library/SKILLS.md and grep -n "docs/library/skills/S-LEARN-" docs/library/INDEX.md (Note: Worker must grep the NEW registry docs/ops/registry/SKILLS.md instead of INDEX.md).

Artifact bundle (required directory)

- storage/handoff/

Proof bundle checklist (required; naming pattern over exact names)

- storage/handoff/DP-OPS-0033-RESULTS.md
- storage/handoff/OPEN-PORCELAIN-work-registry-refactor-0033-[hash].txt
- storage/handoff/OPEN-work-registry-refactor-0033-[hash].txt
- storage/dumps/dump-full-...
- git diff --name-only
- git diff --stat
- NEXT: one single action (include in RESULTS)

MANDATE: No receipt/proof bundle = **AUTOMATIC DISAPPROVAL**.

Closeout routine receipts not being produced CONSISTENTLY INCLUDING PORCELAIN, DUMP AND OPEN is a failure condition.

Partial receipts (e.g., missing OPEN artifacts when a session was started) are considered failures.

REQUIRED OUTPUT

MUST include a "Status" block at the top of RESULTS:

- Scope summary: (1 sentence validating the loop)

- Tracked change: (What was edited)
- Verification: (Output of proofs)

RESULTS file must include (minimum)

- Verification results (RUN/NOT RUN + reason + risk)
- Patch/diff proof bundle (status/diff list or direct diff)
- Receipt pointers (OPEN + PORCELAIN + DUMP outputs/paths)
- git diff --name-only output
- git diff --stat output
- NEXT: one single action

Storage artifact flow (local-only, untracked):

- OPEN and OPEN-PORCELAIN outputs land in storage/handoff/.
- Dump bundles land in storage/dumps/.
- Results receipts land in storage/handoff/DP-OPS-0033-RESULTS.md.
- Operator stores DP drafts under storage/dp/intake/ and moves or copies to storage/dp/processed/ after completion (or when reusing a previously executed DP).
- These are retention paths only; they are not worker prerequisites unless the DP explicitly says so.

---

4. Closeout (Mandatory)

Worker closeout duties (end of session)

- [ ] Produce/Update: storage/handoff/DP-OPS-0033-RESULTS.md
- [ ] If canon/governance surfaces were changed, include **SoP.md** in scope and update it in the same PR slice.
- [ ] Run ops/bin/prune (hygiene).
- [ ] Run ops/bin/llms (refresh context bundles).
- [ ] Ensure proof bundle checklist is satisfied (above).
  (No bundle = DISAPPROVE.)
- [ ] Fill out the Mandatory Closing Block (see below).
- [ ] Append a **TASK Work Log** entry (below) capturing:
  - what changed (high level)
  - gates/verification outcome (pass/fail or not run + reason)
  - blockers (if any)
  - **NEXT:** one single action

Mandatory Closing Block (Varied Wording Required)

Varied Wording provision: Each entry must use meaningfully distinct wording;

copy or minor tense changes are not acceptable. Entry 4 must differ from Entry 1.

1. Primary Commit Header (plaintext)
2. Pull Request Title (plaintext)
3. Pull Request Description (markdown)
4. Final Squash Stub (plaintext) (Must differ from #1)
5. Extended Technical Manifest (plaintext)
6. Review Conversation Starter (markdown)

Integrator/operator closeout duties

- [ ] Review diff vs DP scope/forbidden zones
- [ ] Run repo-gates / required checks
- [ ] Commit/push/open PR/merge (operator-controlled)

4.1 Thread Transition (Reset / Archive Rule)

Purpose: Prevent TASK from becoming a museum while preserving continuity.

When a DP is **complete** (merged) or **ended** (canceled/superseded):

1. Add a final Work Log entry that starts with: THREAD END: and includes:
   - outcome (merged/canceled/superseded)
   - pointer to the DP RESULTS file
   - NEXT: the single action (often “start new DP” or “await operator”)
2. Reset sections **2. Active Context** and **3. Current Dispatch Packet** to the next DP.
3. Keep the Work Log, but separate threads with a visible divider line:
   - ---
   - THREAD START: DP-OPS-

(If the Work Log gets too long, it’s acceptable to keep only the most recent thread’s entries here and move older thread logs into the DP RESULTS file—only when explicitly directed.)

---

5. Work Log (Timestamped Continuity)

> **NOTICE:** This log is for the *active* DP only.
>
> Historical logs are archived in 'storage/archives/'. Do not rely on this log for long-term history.
>
> **Rule:** This is not a transcript. It is the durable breadcrumb trail.
>
> **Expectation:** Append a new timestamped entry on every DP closeout (no

exceptions).

Each entry ends with: NEXT: <one single action>.

- ** — DP-OPS-0033: Registry refactor initialized. Verification: PENDING. Blockers: none. NEXT: Execute registry migration.

Works cited

1. dump-full-main-57af7cdbf.txt
2. ScriptSmith: A Unified LLM Framework for Enhancing IT Operations via Automated Bash Script Generation, Assessment, and Refinement, accessed February 7, 2026, https://ojs.aaai.org/index.php/AAAI/article/view/35147/37302